

RaftKV -- A Dynamically Reconfigurable Distributed Key-Value Store

Project Proposal for UCS503P

Submitted to: Dr. Raghav B. Venkataramaiyer

Thapar Institute of Engineering and Technology

Kunwar Aran Srivastava, CSED
Anubhav Yadav, CSED

Yuvraj Kumar, CSED
Pratik Batish, CSED

August 21, 2026

1 Higher-order goal

...secondary framing

This project aims to deepen understanding of distributed-systems theory -- specifically consensus and fault-tolerant replication -- by building a correct, from-scratch implementation of the Raft consensus algorithm inside a well-engineered Go codebase. While the principles of distributed consensus underpin critical production infrastructure (etcd, CockroachDB, Consul, Kubernetes coordination), the immediate engineering objective is to translate those principles into a testable, modular, and demonstrably correct key-value store that four students can fully understand, not merely assemble from opaque libraries.

RaftKV is explicitly not an attempt to build a production-grade or etcd-equivalent system. Its purpose is to demonstrate correct application of the Raft protocol -- leader election, log replication, dynamic membership, crash recovery -- inside a disciplined software project with automated testing, fault injection, CI/CD, and thorough documentation, built at approximately 10 hours per person per week over a 10-12 week schedule.

2 Time-to-value

...primary heuristic

- We will deliver meaningful, demonstrable increments at the end of each weekly milestone by sequencing work so that each week produces a testable, running artifact -- not just code that compiles but cannot be exercised.
- We will prioritize correctness over feature breadth by adopting a strict MUST / SHOULD / MAY scope classification (S9), ensuring the core Raft consensus and KV operations are solid before any optional feature is attempted.
- We will use CI-backed automation from week 1 -- every pull request triggers build, vet, lint, unit tests, and race detection (go test -race) -- so that integration problems surface immediately, not during a final-week scramble.
- Success at each milestone is defined by what is provably correct (invariant-holding, race-free, crash-recoverable), not merely by what can be demoed on the happy path. The MVP (S9.1) is scoped to be achievable within the first 6 weeks, with subsequent weeks dedicated to membership, fault testing, and hardening.

3 Problem Statement

Single-node key-value stores have no fault tolerance: a crash or restart loses availability entirely, and there is no mechanism to survive machine failure without external replication. This is a fundamental limitation for any system that needs to remain available and consistent in the face of infrastructure failures.

Naively replicating writes to multiple nodes without a consensus protocol risks split-brain scenarios, lost writes, or diverging replicas under network partitions or node crashes. Common pain points include:

- **No crash tolerance:** a crash renders the entire store unavailable. Recovery requires manual intervention and risks data loss for any state that was not durably persisted.
- **Split-brain under partition:** without a formal consensus mechanism, two network partitions may independently accept conflicting writes, leading to irreconcilable state divergence when the partition heals.
- **No dynamic membership:** traditional static-cluster designs require the entire cluster to be stopped and reconfigured to add or remove a node -- a significant operational burden and availability gap.
- **Unsafe crash recovery:** without checksummed, fsync'd durable storage (write-ahead logging), a node that crashes mid-write may recover into an inconsistent state -- partially applied operations, corrupted metadata, or a log that disagrees with what the node had previously acknowledged.

Why this matters now (contextual relevance): The Raft consensus algorithm (Ongaro & Ousterhout, 2014) was designed specifically to be more understandable than Paxos while providing equivalent safety guarantees. It is the foundation of widely deployed production systems (etcd, Consul, TiKV, CockroachDB). Building a correct implementation from scratch -- and validating it under injected faults -- is one of the most effective ways to internalize distributed-systems principles that are otherwise purely theoretical in a coursework setting.

4 Proposed Solution

...Software Proposition

4.1 Overview

Build a from-scratch, dynamically reconfigurable, strongly consistent distributed key-value store (RaftKV) with the following components:

- **Raft consensus core:** Leader election with randomized timeouts, log replication via AppendEntries RPCs, term-based voting, commit-index tracking with majority-quorum semantics, and persistent Raft metadata for crash recovery.
- **Replicated KV state machine:** A deterministic state machine that applies committed log entries as PUT/GET/DELETE operations against an in-memory key-value map, with snapshot and restore capabilities.

- **Dynamic cluster membership:** An operator-driven join protocol where new nodes enter as non-voting learners, catch up to the leader's committed log via AppendEntries (or InstallSnapshot for large gaps), and are promoted to voters only once sufficiently caught up.
- **Durable storage engine:** A write-ahead log (WAL) with checksummed, fsync'd records for Raft metadata and log entries, plus periodic snapshots of the KV state for log compaction and efficient new-node catch-up.
- **Client-facing API and CLI:** An HTTP/JSON API for PUT/GET/DELETE operations and cluster management, plus a raftctl CLI for operator workflows (cluster status, join, remove).
- **Fault-injection and testing framework:** A scriptable harness for crash, partition, delay, and message-drop injection, validating Raft safety invariants under realistic failure scenarios.

4.2 Core workflow

1. Client submits a request (PUT, GET, or DELETE) via HTTP to any node in the cluster.
2. **Leader check:** If the receiving node is not the current leader, it responds with a structured redirect (409 Conflict + leaderHint JSON body) pointing the client to the leader's address.
3. **Leader proposes:** The leader wraps the client's operation into a Raft log entry (with the current term and next log index) and appends it to its local log.
4. **Log replication:** The leader sends the new entry to all followers and learners via gRPC AppendEntries RPCs. Each follower validates consistency (prevLogIndex/prevLogTerm), appends to its WAL (fsync'd before responding), and acknowledges.
5. **Quorum commit:** Once the leader receives acknowledgments from a majority of current voting members (not including learners), it advances its commit index.
6. **State-machine application:** Each node applies committed entries to its KV state machine in index order. The leader then responds to the client with the result.
7. **Membership changes:** A new node sends a JoinCluster RPC to any existing node (redirected to leader). The leader adds it as a learner, replicates until caught up, then promotes to voter via a config-change entry committed under the old quorum.

4.3 Operational constraints for an educational setting

- **Single-key operations only:** No multi-key transactions, no sharding, no partitioned keyspaces.
- **Trusted network:** No TLS, no authentication, no authorization in mandatory scope. Documented as a known limitation.
- **Correctness over performance:** No throughput SLAs. Single-digit to low-tens-of-millisecond commit latency on a 3-5 node Docker cluster is a sanity bound, not a guarantee.
- **Local/Docker deployment only:** Docker Compose for multi-node clusters. Kubernetes,

cloud deployment, and multi-host orchestration are explicitly out of scope.

5 Solution Approach

...Engineering focus

This is an engineering course project. The focus is on correct implementation of a well-understood distributed-systems algorithm (Raft) and professional software engineering practices, not on novel algorithm research.

5.1 Raft consensus implementation

...core algorithm

The Raft core module (`internal/raft`) implements the full Raft state machine with three roles -- Follower, Candidate, and Leader -- plus a learner membership attribute layered on the Follower state. Learners run the same replication logic but never vote and are never counted toward quorum.

Leader election: Nodes start as followers. If a follower receives no heartbeat within a randomized election timeout (150-300ms range, per Raft paper guidance), it transitions to candidate, increments its term, votes for itself, and sends `RequestVote` RPCs to all peers. A candidate wins if it receives votes from a majority of voters; it steps down to follower if it discovers a higher term. At most one leader can exist per term (Election Safety invariant S1), enforced by the persisted `votedFor` field -- a node votes at most once per term.

Log replication: The leader appends client commands as log entries and replicates them via `AppendEntries` RPCs. Each RPC includes `prevLogIndex` and `prevLogTerm` for the Log Matching consistency check (invariant S3): a follower rejects an append if its log doesn't match at the specified position, and the leader retries with earlier entries until consistency is established. Conflict resolution truncates divergent follower entries. The leader never overwrites its own log (Leader Append-Only, invariant S2).

Commit and apply: An entry is committed once a majority of current voters have acknowledged it. The leader advances its `commitIndex` and communicates it to followers via subsequent `AppendEntries` RPCs. Each node applies committed entries to the KV state machine in strict index order via a deterministic `Apply(entry)` interface -- ensuring State Machine Safety (invariant S5).

Persistence: `currentTerm`, `votedFor`, and all log entries are durably persisted (WAL + `fsync`) before any response that implies their durability is sent. This is the foundation of crash recovery: a restarted node reloads its persisted state and rejoins as a follower.

5.2 System architecture

...modular design

RaftKV is decomposed into nine modules with strict dependency rules enforced by code review:

- Raft Core (`internal/raft`): Election, replication, commit, persistence.
- Storage Engine (`internal/storage`): WAL, snapshots, crash recovery, compaction.
- Membership Manager (`internal/membership`): Join, learner tracking, promotion, removal.
- Transport Layer (`internal/transport`): gRPC client/server, Protobuf serialization.
- KV State Machine (`internal/state`): Deterministic Apply for PUT/GET/DELETE.
- Client API (`internal/api`): HTTP handlers, leader-redirect logic.

- CLI (cmd/raftctl): Thin wrapper over HTTP API.
- Fault Injection (internal/fault + tests/): Crash/partition/delay injection hooks.
- Observability (internal/observability): Structured logging, Prometheus metrics.

Coupling rules: The Raft core defines Transport and StateMachine interfaces (dependency inversion) -- it never imports transport, API, or membership packages directly. The KV state machine has no knowledge of Raft, networking, or storage. The transport layer implements the Raft-defined Transport interface and imports only generated Protobuf code. These rules prevent a "big ball of mud" and ensure every module is testable in isolation.

5.3 Dynamic membership

...cluster reconfiguration

Configuration changes (add learner, promote to voter, remove member) are represented as special entries in the same replicated Raft log used for KV commands, committed via the same quorum mechanism. This reuses all existing replication/commit machinery.

Join protocol:

1. New node sends JoinCluster(nodeID, address) to any existing node, which redirects to the leader.
2. Leader validates: no change already in progress, nodeID not already a member.
3. Leader appends a CONFIG_ADD_LEARNER entry to the log and replicates it.
4. Once committed, the new node is a known learner; the leader begins replicating to it.
5. If the learner's log is far behind, the leader sends its latest snapshot (InstallSnapshot) followed by AppendEntries for the remaining tail.
6. Once the learner's matchIndex is within a configurable threshold of the leader's commitIndex, the leader proposes a promotion entry, committed under the old quorum.

Node removal (SHOULD-scope, timeboxed): An operator issues raftctl cluster remove <nodeID>. The leader validates the removal won't drop below minimum quorum size, proposes a CONFIG_REMOVE entry, and commits it under the old voter set. Self-removal of the current leader is disallowed in this scope.

5.4 Storage and durability

...crash recovery

Write-ahead log (WAL): All Raft metadata and log entries are persisted as checksummed, append-only records in a single WAL file per node. Each record contains a length field (for torn-write detection), a CRC checksum, a record type (STATE or LOG_ENTRY), and the serialized payload. The critical write-path invariant: append -> fsync -> return success. No response implying durability may be sent before the relevant fsync completes.

Crash recovery: On startup, the WAL is sequentially replayed. On the first checksum failure or truncated frame, reading stops -- everything before that point is the recovered log. Recovery is idempotent: a crash during recovery itself is safe because recovery reads persisted, checksummed state without mutating it until fully validated.

Snapshots and compaction: The KV state machine is periodically snapshotted (triggered every N committed entries, configurable). Each snapshot records lastIncludedIndex, lastIncludedTerm, the serialized KV map, and a checksum. After a snapshot is durably written, WAL entries up to that

point are discarded (compaction), preventing unbounded WAL growth. Snapshot transfer to lagging or new nodes uses a single-message InstallSnapshot RPC.

5.5 CI/CD and delivery pipeline

GitHub Actions (.github/workflows/ci.yml) runs on every pull request and push to main:

1. go build ./... -- compilation check
2. go vet ./... -- static analysis
3. golangci-lint run -- code quality
4. go test ./... -- unit and integration tests
5. go test -race ./... -- data-race detection (non-negotiable)

Docker image build as a CI job produces a raftkv container image, validated by starting a Docker Compose cluster and hitting /cluster/status. Git workflow: main is always green. Feature branches named area/short-description. Pull requests required for all merges with minimum 1 reviewer; 2 reviewers recommended for internal/raft and internal/membership.

6 Evaluation Criterion

...measurable and attributable

We define success using measurable properties tied to the core correctness and engineering quality of the system. A consensus system's value proposition is safety under failure.

6.1 Primary evaluation metric

Raft Safety Invariant Compliance: All seven safety invariants (S1-S7) hold under fault injection across all test scenarios.

- **S1 -- Election Safety:** at most one leader per term. Measured via property test across hundreds of randomized election runs.
- **S2 -- Leader Append-Only:** leader never overwrites its own log. Verified via unit test + integration assertion.
- **S3 -- Log Matching:** same index+term implies identical prefix. Verified by comparing logs across all nodes.
- **S4 -- Leader Completeness:** committed entries present in all future leaders' logs. Verified via fault test: kill leader after commit, verify new leader has the entry.
- **S5 -- State Machine Safety:** no two nodes apply different entries at the same index. Verified by comparing applied KV state across all nodes after writes.
- **S6 -- Membership-Change Safety:** config change only effective once committed; learner never in quorum.
- **S7 -- No commit without quorum:** entry committed only after majority voter ack. Verified via partition test: write to minority side, assert it never commits.

Target: Zero invariant violations across the full fault-injection test suite.

6.2 Secondary evaluation metrics

- **Race-detector cleanliness:** go test -race ./... passes with zero data races, consistently. Binary

pass/fail, non-negotiable.

- **Crash-recovery correctness:** A crashed-and-restarted node converges to the cluster's committed state without data loss or corruption.
- **Fault-injection coverage:** The test suite covers leader crash, follower crash, network partition, node restart, and join during concurrent writes.
- **Join protocol completeness:** A new node can join, catch up, be promoted, and participate in subsequent elections.
- **Test tier coverage:** Unit, integration, Docker, fault-injection, and membership test tiers all populated.

6.3 Validation plan

...testing-driven

Validation is structured in five tiers:

- **Unit:** Single-module logic in isolation: election vote-granting rules, log-matching, WAL checksum, snapshot serialize/restore.
- **Integration (in-process):** Multiple Raft instances in one test binary: 3-node and 5-node election, replication, PUT/GET/DELETE end-to-end.
- **Integration (Docker):** Full multi-container cluster bring-up, real container joins, docker kill leader and observe re-election.
- **Fault injection:** Leader crash, follower crash, multiple failures below quorum, network partition, delayed/dropped packets.
- **Membership:** Join during concurrent writes, learner catch-up, promotion, restart during catch-up.

Property-style correctness checks run across many randomized fault-injection scenarios, asserting S1-S7. Required CI commands: `go test ./...` and `go test -race ./...`

7 Scalability

...with foresight

The proposition should scale in both theoretical and practical senses, within the bounded scope of an educational project.

1. **Scalable (theoretically):**
 - Configurable N-node clusters -- cluster size is set at bootstrap time, not hardcoded. Quorum calculation ($\text{floor}(N/2)+1$) adapts automatically.
 - Dynamic node addition via an explicit join protocol without stopping the cluster.
 - Snapshot-based catch-up for new or lagging nodes, avoiding full log replay.
 - Stateless client API (HTTP/JSON) with no sticky sessions.
2. **Operationally deployable (practically):**
 - Docker Compose for N-node local clusters, each node as an independent container with its own named volume.
 - CI/CD pipeline producing container images and running automated tests.
 - No external dependencies beyond Go, gRPC, and Docker.

8 Engine availability heuristic

A mature set of engines (tooling, frameworks, and services) is available to implement this project, enabling the team to focus on consensus correctness rather than building infrastructure:

- **Language:** Go 1.22+: excellent concurrency primitives, built-in race detector, dominant language for Raft implementations (etcd, Consul, HashiCorp Raft).
- **Node-to-node transport:** gRPC + Protocol Buffers: strongly typed RPC contracts, built-in streaming, mature Go tooling (protoc-gen-go-grpc).
- **Client API:** HTTP/JSON (net/http): universal client compatibility, simple to test with curl.
- **Durable storage:** Custom WAL + filesystem: no external DB dependency; the WAL is a core deliverable.
- **Containerization:** Docker + Docker Compose: industry-standard, supports named volumes and network isolation.
- **CI/CD:** GitHub Actions: free for public repos, native Go support, integrated with PR workflow.
- **Observability:** Structured JSON logging + Prometheus text format: lightweight, no external backend.
- **Testing:** Go's testing package + race detector: built-in, zero-dependency, integrates with CI.
- **Documentation:** Markdown + Mermaid diagrams + ADRs: renderable in GitHub, version-controlled.

We will use these mature components to focus on workflow design, Raft correctness, and engineering measurement rather than inventing new infrastructure.

9 Project scope and deliverables

...iteration-friendly

Scope is controlled via a strict MUST / SHOULD / MAY / OUT_OF_SCOPE classification. MUST items form the MVP. SHOULD items are attempted only after the MVP is stable.

9.1 Initial deliverable

...MVP (Weeks 1-6)

- Week 1: Requirements ratification, repo setup, branching model, module interfaces, .proto skeletons, Mermaid diagrams, coding standards, Raft fundamentals study.
- Week 2: Node lifecycle, Raft states, election timers, basic RPC scaffolding.
- Week 3: Leader election, heartbeats, voting, terms, election unit tests.
- Week 4: Log replication, AppendEntries, commit index, majority commit, log-matching/conflict resolution, integration tests.
- Week 5: KV state machine (PUT/GET/DELETE), WAL persistence, basic crash recovery.
- Week 6: HTTP API, CLI, follower-redirect handling, Docker Compose 3-node cluster, end-to-end integration testing.

MVP feature set (all MUST):

- Configurable N-node cluster with correct leader election and log replication.
- PUT/GET/DELETE via HTTP with leader-redirect for non-leader writes.
- Durable WAL + crash recovery (checksummed, fsync'd).

- Basic snapshotting + log compaction.
- gRPC transport layer for all node-to-node RPCs.
- go test -race ./... clean. CI running build/vet/test/race on every PR.
- Core documentation: ratified SDD, API docs, Mermaid diagrams.

Team responsibility (primary ownership, not isolated silos):

- Person 1: Raft Core (election, replication, commit). Cross-review from Person 2, Person 4.
- Person 2: Storage + Membership (WAL, snapshots, join). Cross-review from Person 1, Person 4.
- Person 3: Networking + API (gRPC, HTTP, CLI). Cross-review from Person 1, Person 2.
- Person 4: Testing + Reliability + DevOps (fault injection, CI/CD, Docker). Cross-review from all.

9.2 Subsequent deliverables

...Weeks 7-12

- Week 7: Dynamic membership: join protocol, learner role, node identity, catch-up mechanism.
- Week 8: Snapshot transfer (InstallSnapshot), learner promotion, node removal attempt (SHOULD -- timeboxed), membership fault testing.
- Week 9: Crash-recovery hardening, fault-injection framework, partition simulation, race detection sweep, consistency testing.
- Week 10: Observability polish (metrics endpoint, structured logging), health checks, basic benchmarking (MAY, timeboxed).
- Week 11: Hardening, bug fixing, CI/CD polish, documentation catch-up -- treated as flex/buffer capacity.
- Week 12: Final testing, architecture doc finalization, demo scenarios, performance notes, final report, demo preparation.

Stretch goals (in priority order, only after MVP is stable):

1. **Node removal (SHOULD):** Safe voter removal, including the leader-self-removal edge case.
2. **Read-index / linearizable reads (SHOULD):** Closes the known stale-read consistency gap.
3. **Chunked/streaming snapshot transfer (MAY):** For very large KV state.
4. **Deeper metrics (MAY):** Commit latency histograms, catch-up lag gauges.
5. **Performance benchmarking (MAY):** Suite + write-up.
6. **Watches, TTL/leases, CAS, transactions (MAY):** Unlikely to fit schedule.

Hard rule: None of the stretch goals may begin until the MVP items are implemented, tested, and documented.

10 Risks and mitigations

- **R1:** Log replication/conflict resolution takes longer than planned (week 4) -- historically the single hardest correctness-critical chunk of Raft. Mitigation: explicit buffer in roadmap; defer non-critical polish to protect this week.

- **R2:** Membership protocol underestimated (weeks 7-8) -- depends on Raft core being solid. Mitigation: hard sequencing dependency; do not start membership until election + replication tests are green.
- **R3:** Fault-injection harness built too late to validate earlier weeks. Mitigation: build minimal harness (process kill + message drop) as early as week 3-4.
- **R4:** Under-investment in tests relative to "getting it to demo." Mitigation: Definition of Done explicitly requires failure tests; code review checklist references S1-S7 invariants.
- **R5:** Scope creep toward "make it more like etcd." Mitigation: strict OUT_OF_SCOPE list and MUST/SHOULD/MAY classification re-read before adding anything.
- **R6:** Uneven distributed-systems background across 4-person team. Mitigation: weekly integration sessions; mandatory cross-review on Raft/membership PRs.
- **R7:** Underestimating documentation/report-writing time. Mitigation: week 12 budgeted; ADRs written incrementally throughout, not batched at the end.

Schedule compression policy: If compressed to 8-9 weeks, cut in this order: (1) node removal entirely, (2) read-index reads, (3) metrics beyond 5 MUST-scope gauges, (4) fault-test breadth (keep only leader-crash + partition). Never cut: election/replication/commit correctness and its tests, WAL durability tests, the race-detector requirement, or the join/learner/promotion happy path.

11 Summary

This project proposes a from-scratch, dynamically reconfigurable, strongly consistent distributed key-value store built on the Raft consensus algorithm in Go. It meets the UCS503P course criteria by:

- **Emphasizing time-to-value** through weekly iteration milestones, with a well-scoped MVP (leader election, log replication, KV operations, WAL persistence, crash recovery) deliverable within the first 6 weeks, followed by dynamic membership, fault injection, and hardening.
- **Implementing CI/CD from day 1** -- automated build, vet, lint, test, and race detection on every pull request -- to support frequent, safe integration of a distributed system where subtle concurrency and consistency bugs are the primary engineering risk.
- **Using measurable evaluation criteria** -- Raft safety invariants (S1-S7) as the primary metric, validated by a structured fault-injection test suite covering leader crash, network partition, crash recovery, and dynamic membership under concurrent writes.
- **Remaining focused on engineering correctness** -- not production scale, feature parity with etcd, or research-driven novelty. The system is bounded (single-key operations, no sharding, no auth, Docker-only) to keep scope achievable for 4 students at ~10 hours/week over 10-12 weeks.

The full feasibility assessment (documented in the Software Design Document, S47) acknowledges that the MUST + SHOULD scope (~600 person-hours) exceeds the effective capacity (~300-360 hours for 10-12 weeks). The proposal addresses this honestly: the MVP is scoped to MUST items only, SHOULD/MAY items are strictly prioritized stretch goals, and a clear cut-order is defined for schedule compression. This project delivers a real distributed system, not a toy -- but a deliberately bounded one.

*Roll No: 1024240003, Email: ksrivastava_be24@thapar.edu

+Roll No: 1024240143, Email: ykumar2_be24@thapar.edu

#Roll No: 1024240104, Email: ayadav6_be24@thapar.edu

\$Roll No: 1024240101, Email: pbatish_be24@thapar.edu